

Overview

As agentic AI systems grow, interoperability becomes a first-class engineering problem. Teams increasingly need: - reliable tool/context integration across apps and data systems, - agent-to-agent collaboration across organizational boundaries, - explicit safety and authorization boundaries.

This chapter covers two important standards used in that interoperability layer: - **MCP (Model Context Protocol)** for model-to-tools/resources/prompts integration. - **A2A (Agent2Agent Protocol)** for agent-to-agent collaboration.

A practical way to remember the relationship: - MCP helps an agent connect to tools and context. - A2A helps one agent coordinate with another agent.

MCP Foundations

What MCP standardizes

MCP standardizes how an AI host/client discovers and invokes capabilities exposed by servers.

Key primitives: - **Tools**: executable functions invoked by the model/client. - **Resources**: data/context the model can read. - **Prompts**: reusable interaction templates.

The protocol is JSON-RPC based and supports lifecycle/capability negotiation and notification patterns.

MCP architecture

Core participants: - **Host**: the AI application runtime. - **Client**: host-side connection for one server. - **Server**: capability provider (local or remote).

Transport models: - **stdio** for local process communication, - streamable HTTP for remote multi-client deployments.

MCP operation flow

1. Host initializes client-server session.
2. Client discovers capabilities (**tools/list**, **resources/list**, etc.).
3. Model/agent selects a tool based on prompt context.
4. Host invokes **tools/call** with validated arguments.
5. Output is returned and incorporated into response planning.

MCP safety implications

MCP tooling is powerful and must enforce: - tool-level allowlists, - user-visible confirmation for sensitive actions, - auth and rate limits, - audit logging for tool calls and outputs.

A2A Foundations

What A2A standardizes

A2A standardizes communication between independent agentic applications. It targets interoperability when agents are opaque internally and may run in different environments.

A2A focuses on: - capability discovery, - modality negotiation (text/files/structured payloads), - task lifecycle and asynchronous collaboration, - secure scoped access.

A2A collaboration model

A2A workflows are generally task oriented: 1. Agent A discovers Agent B capabilities. 2. Agent A submits task request. 3. Agent B processes asynchronously. 4. Agent A retrieves updates by polling/streaming/push. 5. Agents may continue multi-turn coordination until task terminal state.

This is different from single-request tool calls because long-running task semantics are explicit.

Authorization and boundaries

A2A implementations must enforce authorization scope for each request and each task retrieval operation.

Typical scope dimensions: - user identity, - role/project membership, - tenant boundary, - custom policy rules.

MCP vs A2A: How to choose

Use **MCP** when: - your primary need is exposing tools/resources to one model host, - you need tight integration with IDE/assistant/tool ecosystems, - you are composing context and callable actions.

Use **A2A** when: - independent agents must coordinate across services or organizations, - tasks are long-running and asynchronous, - collaboration requires explicit task lifecycle and modality negotiation.

Use both together when: - each agent uses MCP internally for tools/context, - agents communicate externally through A2A for orchestration.

Reference Architecture Pattern

A practical enterprise pattern: 1. **Edge Orchestrator Agent** receives user goal. 2. It uses MCP to access internal systems (docs, DBs, workflow APIs). 3. For specialized tasks, it delegates to external partner/domain agents over A2A. 4. Returned artifacts are validated, merged, and presented with provenance.

This gives local execution control plus cross-agent interoperability.

Failure Modes and Mitigations

Failure mode 1: Protocol confusion

Symptom: Treating A2A like a low-latency tool call. Mitigation: Introduce task state model with timeouts and retries.

Failure mode 2: Excessive trust in remote agent output

Symptom: Blindly applying delegated outputs. Mitigation: Validate outputs with schema checks, policy checks, and optional human review.

Failure mode 3: Tool misuse escalation

Symptom: Agents call high-risk tools without proper approvals. Mitigation: permission tiers, approval checkpoints, and immutable audit trails.

Failure mode 4: Unbounded orchestration loops

Symptom: Agent ping-pong between systems. Mitigation: step budgets, termination criteria, and fallback to human escalation.

Business Case Studies & Exceptions

Case 1: Enterprise support triage mesh

An internal support agent uses MCP to fetch ticket history and run diagnostics. It uses A2A to hand off billing disputes to a finance specialist agent managed by another team.

Exception: For simple single-team deployments, A2A may be unnecessary overhead; MCP plus one orchestrator is enough.

Case 2: Supply chain partner coordination

A retailer agent coordinates with supplier agents for inventory reconciliation. A2A handles asynchronous task exchange; MCP is used inside each organization for local ERP access.

Exception: If legal constraints prevent cross-agent API trust, outputs must be reduced to signed document exchange.

Case 3: Security operations assistant

A SOC assistant uses MCP to query logs and incident systems. For malware intelligence, it queries a partner threat-analysis agent via A2A with strict scoped tokens.

Exception: High-severity response actions still require human approval gates.

Interview Questions & Answers

1. **Q:** What does MCP solve?
A: Standardized model-host integration with tools, resources, and prompts.
2. **Q:** Main MCP primitives?
A: Tools, resources, prompts.
3. **Q:** What does A2A solve?
A: Interoperable communication between independent agentic systems.
4. **Q:** MCP vs A2A in one line?
A: MCP is model-to-capability plumbing; A2A is agent-to-agent collaboration.
5. **Q:** Why is A2A asynchronous by design?
A: Many agent tasks are long-running and require lifecycle tracking.
6. **Q:** Why separate host/client/server roles in MCP?
A: To allow one host to safely manage multiple independent capability providers.
7. **Q:** What is capability discovery?
A: Protocol-level listing of what tools/tasks an endpoint can perform.
8. **Q:** Key authorization requirement in A2A?
A: Scope every operation to caller's authorized boundary.
9. **Q:** Why keep human-in-the-loop for sensitive tools?
A: To prevent unsafe autonomous execution and policy violations.
10. **Q:** Can MCP replace A2A?
A: Not fully; MCP is not a full cross-agent task-collaboration protocol.
11. **Q:** Can A2A replace MCP?
A: Not fully; A2A does not replace local model tool/resource integration patterns.
12. **Q:** What is a safe interoperability baseline?
A: Schema validation, scoped auth, approval gates, retries/timeouts, and audit logs.
13. **Q:** Common integration anti-pattern?
A: Exposing high-risk tools without explicit policy and approval control.
14. **Q:** How do you prevent orchestration loops?
A: Step budgets, terminal conditions, and escalation rules.

15. **Q:** Why log protocol-level events?
A: For debugging, governance, and incident forensics.
16. **Q:** How should remote agent outputs be treated?
A: As untrusted until validated against schema/policy/business rules.
17. **Q:** What is modality negotiation?
A: Agreeing on interaction payload types (text/files/structured data).
18. **Q:** Where does MCP fit in IDE copilots?
A: As a standardized way to extend assistants with external tools and context.
19. **Q:** Where does A2A fit in multi-org workflows?
A: As a protocol for interoperable agent collaboration without exposing internals.
20. **Q:** One production recommendation?
A: Treat interoperability protocols as security boundaries, not just integration convenience.

References

- MCP architecture: <https://modelcontextprotocol.io/docs/learn/architecture>
- MCP tools spec: <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- MCP specification repo: <https://github.com/modelcontextprotocol/modelcontextprotocol>
- A2A specification overview: <https://a2a-protocol.org/dev/specification/>
- A2A protocol repo: <https://github.com/a2aproject/A2A>